Assignment : 16.1

Batch : 29

Task 1 – Student Information System Schema
Task:
Design a database schema for a Student Information System and
generate queries using AI.
Instructions:
• Tables: Students, Courses, Enrollments.
• Define primary keys, foreign keys, and relationships.
• Generate queries:
o Insert a new student record.
o Fetch all courses enrolled by a student.
o Count number of students in each course.
Expected Output:
• SQL CREATE TABLE statements and queries for student–
course relationships.

prompt:

Design a Student Information System with three tables: Students, Courses, and Enrollments. Use primary keys for each table and foreign keys in Enrollments to create a many-to-many relationship. Perform operations to insert a student, retrieve their courses, and count students per course.

code:

```python
import sqlite3

# Connect to database (creates file if not exists)
conn = sqlite3.connect("student_system.db")
cursor = conn.cursor()

# ==============================
# Create Tables
# ==============================
cursor.execute("""
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    department TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL,
    credits INTEGER
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY,
```

```
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_id INTEGER,
    enrollment_date TEXT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
)
""")

# ==============================
# Insert Data
# ==============================
cursor.execute("""
INSERT INTO Students (student_id, name, email, department)
VALUES (101, 'John Doe', 'john@example.com', 'Computer Science')
""")

# Sample course data
cursor.execute("INSERT INTO Courses VALUES (1, 'DBMS', 4)")
cursor.execute("INSERT INTO Courses VALUES (2, 'Python', 3)")

# Enroll student
cursor.execute("INSERT INTO Enrollments VALUES (1, 101, 1, '2026-03-24')")
cursor.execute("INSERT INTO Enrollments VALUES (2, 101, 2, '2026-03-24')")

# ==============================
# Queries
# ==============================
```

```
# 1. Fetch all courses of a student
print("Courses enrolled by student 101:")
cursor.execute("""
SELECT c.course_id, c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 101
""")
for row in cursor.fetchall():
    print(row)

# 2. Count students in each course
print("\nNumber of students in each course:")
cursor.execute("""
SELECT c.course_name, COUNT(e.student_id)
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
""")
for row in cursor.fetchall():
    print(row)

# Save changes and close
conn.commit()
conn.close()
```

output:

```
Courses enrolled by student 101:
(1, 'DBMS')
(2, 'Python')

Number of students in each course:
('DBMS', 1)
('Python', 1)
```

Justification:
The design uses separate tables to reduce data redundancy and ensure normalization. The Enrollments table handles the many-to-many relationship between students and courses efficiently. Primary keys ensure unique records, while foreign keys maintain data integrity. This structure is scalable, organized, and supports efficient querying.

Task 2 – Hospital Management Database
Task:
Create schema and queries for a Hospital Management System.
Instructions:
• Tables: Doctors, Patients, Appointments.
• Use AI to define constraints (unique IDs, valid dates).
• Generate queries:
o List all appointments for a specific doctor.
o Retrieve patient history by patient ID.
o Count total patients treated by each doctor.
Expected Output:
• Normalized schema and SQL queries with joins.

prompt:

Design a Hospital Management System with Doctors, Patients, and Appointments tables. Use primary and foreign keys with constraints for valid data. Perform queries to list doctor appointments, retrieve patient history, and count patients per doctor.

```python
import sqlite3

# Connect to database
conn = sqlite3.connect("hospital.db")
cursor = conn.cursor()

# ==============================
# Create Tables
# ==============================
cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctors (
    doctor_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    specialization TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER CHECK(age > 0),
    gender TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointments (
    appointment_id INTEGER PRIMARY KEY,
    doctor_id INTEGER.
```

```python
        appointment_date TEXT NOT NULL,
        FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
        FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
    )
    """)

    # ==============================
    # Insert Sample Data
    # ==============================
    cursor.execute("INSERT INTO Doctors VALUES (1, 'Dr. Smith', 'Cardiology')")
    cursor.execute("INSERT INTO Doctors VALUES (2, 'Dr. John', 'Neurology')")

    cursor.execute("INSERT INTO Patients VALUES (101, 'Alice', 25, 'Female')")
    cursor.execute("INSERT INTO Patients VALUES (102, 'Bob', 30, 'Male')")

    cursor.execute("INSERT INTO Appointments VALUES (1, 1, 101, '2026-03-24')")
    cursor.execute("INSERT INTO Appointments VALUES (2, 1, 102, '2026-03-25')")
    cursor.execute("INSERT INTO Appointments VALUES (3, 2, 101, '2026-03-26')")

    # ==============================
    # Queries
    # ==============================

    # 1. List all appointments for a specific doctor
    print("Appointments for Doctor ID 1:")
    cursor.execute("""
    SELECT a.appointment_id, p.name, a.appointment_date
    FROM Appointments a
    JOIN Patients p ON a.patient_id = p.patient_id
    WHERE a.doctor_id = 1
```

```python
for row in cursor.fetchall():
    print(row)

# 2. Retrieve patient history by patient ID
print("\nPatient history for Patient ID 101:")
cursor.execute("""
SELECT a.appointment_id, d.name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 101
""")
for row in cursor.fetchall():
    print(row)

# 3. Count total patients treated by each doctor
print("\nTotal patients treated by each doctor:")
cursor.execute("""
SELECT d.name, COUNT(DISTINCT a.patient_id)
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name
""")
for row in cursor.fetchall():
    print(row)

# Save and close
conn.commit()
conn.close()
```

output:

```
Appointments for Doctor ID 1:
(1, 'Alice', '2026-03-24')
(2, 'Bob', '2026-03-25')

Patient history for Patient ID 101:
(1, 'Dr. Smith', '2026-03-24')
(3, 'Dr. John', '2026-03-26')

Total patients treated by each doctor:
('Dr. John', 1)
('Dr. Smith', 2)
```

Justification:

- The schema is **normalized** to avoid redundancy.
- **Appointments table** links Doctors and Patients (many-to-many).
- **Primary keys** ensure uniqueness.
- **Foreign keys** maintain relationships and data integrity.
- **Constraints (CHECK, NOT NULL)** ensure valid data (e.g., age, date)

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

Prompt

Design a Library Management System with Books, Members, and Loans tables. Use keys and indexing for performance. Perform queries to get issued books, overdue books, and count books per member.

code:

```python
import sqlite3
from datetime import datetime, timedelta

# Connect to database
conn = sqlite3.connect("library.db")
cursor = conn.cursor()

# ================================
# Create Tables
# ================================
cursor.execute("""
CREATE TABLE IF NOT EXISTS Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    author TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Members (
    member_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Loans (
    loan_id INTEGER PRIMARY KEY,
```

```python
    member_id INTEGER,
    loan_date TEXT,
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
)
""")

# ================================
# Indexing (for faster queries)
# ================================
cursor.execute("CREATE INDEX IF NOT EXISTS idx_book ON Loans(book_id)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_member ON Loans(member_id)")

# ================================
# Insert Sample Data
# ================================
cursor.execute("INSERT INTO Books VALUES (1, 'DBMS', 'Author A')")
cursor.execute("INSERT INTO Books VALUES (2, 'Python', 'Author B')")

cursor.execute("INSERT INTO Members VALUES (101, 'Alice')")
cursor.execute("INSERT INTO Members VALUES (102, 'Bob')")

today = datetime.now().date()
old_date = today - timedelta(days=40)

cursor.execute("INSERT INTO Loans VALUES (1, 1, 101, ?, NULL)", (today,))
cursor.execute("INSERT INTO Loans VALUES (2, 2, 102, ?, NULL)", (old_date,))
```

```python
# 1. Retrieve all books currently issued
print("Books currently issued:")
cursor.execute("""
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
""")
for row in cursor.fetchall():
    print(row)

# 2. Find overdue books (>30 days)
print("\nOverdue books:")
cursor.execute("""
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND julianday('now') - julianday(l.loan_date) > 30
""")
for row in cursor.fetchall():
    print(row)

# 3. Count books loaned by each member
print("\nBooks loaned by each member:")
cursor.execute("""
```

```python
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name
""")
for row in cursor.fetchall():
    print(row)

# Save & close
conn.commit()
conn.close()
```

```
Books currently issued:
('DBMS', 'Alice', '2026-03-24')
('Python', 'Bob', '2026-02-12')

Overdue books:
('Python', 'Bob', '2026-02-12')

Books loaned by each member:
('Alice', 1)
('Bob', 1)
/tmp/ipykernel_6045/2543397561.py:56: DeprecationWarning: The default date adapter is deprecated as of P
  cursor.execute("INSERT INTO Loans VALUES (1, 1, 101, ?, NULL)", (today,))
/tmp/ipykernel_6045/2543397561.py:57: DeprecationWarning: The default date adapter is deprecated as of P
  cursor.execute("INSERT INTO Loans VALUES (2, 2, 102, ?, NULL)", (old_date,))
```

Justification:

- The schema is **normalized** into Books, Members, and Loans tables.
- **Loans table** connects books and members (many-to-many).
- **Indexes** on `book_id` and `member_id` improve query performance.
- **Conditions & joins** efficiently retrieve issued, overdue, and count data.
- Ensures **data integrity, scalability, and fast access**.
-

Task 4 – Real-Time Application: Online Shopping Database
Scenario:
Design a database for an E-commerce platform.
Requirements:
• Tables: Users, Products, Orders, OrderDetails.
• Generate queries to:
o Retrieve all orders by a user.
o Find the most purchased product.
o Calculate total revenue in a given month.
• AI should also suggest normalization improvements and query optimization.
Expected Output:
• Complete schema with relationships + SQL queries for analytics.

prompt:

Design an Online Shopping Database using Users, Products, Orders, and OrderDetails tables. Users store customer data, Products store item details, Orders store purchase records, and OrderDetails link orders with products. Primary and foreign keys maintain relationships. Queries are used to retrieve user orders, find the most purchased product, and calculate monthly revenue. Indexing and normalization improve performance and reduce redundancy, making the system efficient and scalable

code:

```
import sqlite3

conn = sqlite3.connect("ecom.db")
c = conn.cursor()

# Tables
c.execute("CREATE TABLE IF NOT EXISTS Users(user_id INTEGER PRIMARY KEY, name TEXT)")
c.execute("CREATE TABLE IF NOT EXISTS Products(product_id INTEGER PRIMARY KEY, name TEXT, price REAL)")
c.execute("CREATE TABLE IF NOT EXISTS Orders(order_id INTEGER PRIMARY KEY, user_id INTEGER, date TEXT)"
c.execute("CREATE TABLE IF NOT EXISTS OrderDetails(id INTEGER PRIMARY KEY, order_id INTEGER, product_ic

# Sample Data
c.execute("INSERT INTO Users VALUES (1,'Alice')")
c.execute("INSERT INTO Products VALUES (1,'Laptop',50000)")
c.execute("INSERT INTO Orders VALUES (1,1,'2026-03-01')")
c.execute("INSERT INTO OrderDetails VALUES (1,1,1,2,50000)")

# Queries
print("Orders:", list(c.execute("SELECT * FROM Orders WHERE user_id=1")))
print("Most Product:", list(c.execute("SELECT product_id,SUM(qty) FROM OrderDetails GROUP BY product_ic
print("Revenue:", list(c.execute("SELECT SUM(qty*price) FROM OrderDetails")))

conn.commit()
conn.close()
```

```
Orders: [(1, 1, '2026-03-01')]
Most Product: [(1, 2)]
Revenue: [(100000.0,)]
```

Justification :

- The schema is **normalized** to reduce redundancy
- **OrderDetails** manages many-to-many relationships
- Queries use **joins and aggregation** for analytics
- Indexing improves **query speed and performance**
- Suitable for **real-time scalable e-commerce systems**

Task 5 – University Examination Database
Design a database schema for a University Examination System and generate SQL queries using AI.
Instructions:
• Tables: Students, Subjects, Exams, Results.
• Define primary keys, foreign keys, and referential integrity constraints.
• Generate queries:
o Insert examination results for a student.
o Retrieve subject-wise marks for a student.
o Calculate average marks for each subject.
Expected Output:
• Normalized schema and SQL queries involving aggregation and joins.

Prompt:

Create a University Exam System (Python + SQLite) with Students, Subjects, Exams, Results. Use keys & constraints.

Do:
- Insert result
- Get subject marks
- Find average marks

code:

```
import sqlite3

# Use a fresh database file to avoid lock
with sqlite3.connect("university_new.db", timeout=10) as conn:
    c = conn.cursor()

    # Tables
    c.execute("CREATE TABLE IF NOT EXISTS Students(id INTEGER PRIMARY KEY, name TEXT)")
    c.execute("CREATE TABLE IF NOT EXISTS Subjects(id INTEGER PRIMARY KEY, name TEXT)")
    c.execute("CREATE TABLE IF NOT EXISTS Exams(id INTEGER PRIMARY KEY, subject_id INTEGER)")
    c.execute("CREATE TABLE IF NOT EXISTS Results(id INTEGER PRIMARY KEY, student_id INTEGER, exam_id I

    # Data
    c.execute("INSERT INTO Students VALUES (1,'Alice')")
    c.execute("INSERT INTO Subjects VALUES (1,'Math')")
    c.execute("INSERT INTO Exams VALUES (1,1)")
    c.execute("INSERT INTO Results VALUES (1,1,1,85)")

    # Queries
    print(list(c.execute("SELECT s.name, r.marks FROM Results r JOIN Exams e ON r.exam_id=e.id JOIN Sub
    print(list(c.execute("SELECT s.name, AVG(r.marks) FROM Results r JOIN Exams e ON r.exam_id=e.id JOI
```

```
[('Math', 85)]
[('Math', 85.0)]
```

Justification :

- The schema is **normalized** into Students, Subjects, Exams, and Results
- **Results table** links students and exams (many-to-many)
- **Primary & foreign keys** ensure data integrity
- **Constraints** (marks ≥ 0) maintain valid data
- Queries use **joins and aggregation** for efficient analysis